Register No. : Experiment No. : Date :

LABORATORY CONTINUOUS EVALUATION			
S.No.	PROGRAM	Max. Marks	Marks Scored
1.	Design of Algorithm	4	
2.	Viva-Voce on Algorithm	4	
3.	Implementation	4	
4.	Design of Test Cases	4	
5.	Evaluation of Program	4	
	TOTAL	20	

Functionality of the Program is verified on the System

Faculty Signature with Date

Aim:- To deploy the trained customer churn prediction machine learning model using FastAPI and develop REST API end points for single and batch customer churn predictions To implement input validation error handling, model loading, health monitoring, and API testing to ensure the reliability and performance of the prediction service.

Project file structure:-

```
churn-prediction.
├── api
│   ├── __init__.py
│   ├── main.py
│   ├── model_loader.py
│   ├── prepsocessing.py
│   ├── schemas.py
│   └── monitoring_logger.py.
├── deployment
│   ├── model.pkl.
│   ├── preprocessor.pkl.
│   └── model_metadata.json.
├── data
├── models
├── reports
│   ├── api_testing_report.json
│   └── api_performance_report.json
├── tests
│   └── test_api.py
├── conconfig.yaml
├── requirements.txt
└── requirements_api.txt
```

LABORATORY CONTINUOUS EVALUATION			
S.No.	PROGRAM	Max. Marks	Marks Scored
1.	Design of Algorithm	4	
2.	Viva-Voce on Algorithm	4	
3.	Implementation	4	
4.	Design of Test Cases	4	
5.	Evaluation of Program	4	
	TOTAL	20	
Functionality of the Program is verified on the System			
Faculty Signature with Date			

main.py

```
import pandas as pd
from fastapi import FastAPI, HTTP Exception
from api.model_loader import ModelLoader
from api.preprocessing import clean_prediction_data
from api.schemas import CustomerInput
model_loader = ModelLoder()
app = FastAPI ( title = "Churn Prediction API",
                description = "customer churn Prediction",
                version = "1.0.0")

@app.get("/")
def root():
    return { "message : "churn prediction API is running",
             "status" : "success"}
@app.get ("health")
def health():
    return { "status" : "healthy" }
@app.get("/metadata")
def metadata ():
    return { "service" : Churn Prediction API",
             "version" : "1.0.0",
             "model_loaded" : model_loder.model is not None,
             "Preprocessor_loaded" : model_loader.preprocessor
                                     is not None }
@app.post("/Predict")
def predict (data : Customer Input):
    if not model_loader.is ready():
        raise HTTP Exception (status_code = 503,
                              detail = "Model is not available")
df = pd.DataFrame ([data.model_dump()])
df = clean_prediction_data(df)
```

Output:-

uvicorn running on http://127.0.0.1:8000
Application startup complete.
URL: https://127.0.0.1:8000/
{ "message": "churnPrediction API is running",
"lab" : "Lab 10",
"status" : "success" }
URL : https://127.0.0.1:8000/health
{ "status": "healthy" }
URL : https://127.0.0.1:8000/predict
{ "prediction": 1, "churn-probability": 0.73 }

23/23 tests passed

Total test cases	23
Passed	23
Failed	0

overall status in saved PASSED.
report

LABORATORY CONTINUOUS EVALUATION			
S.No.	PROGRAM	Max. Marks	Marks Scored
1.	Design of Algorithm	4	
2.	Viva-Voce on Algorithm	4	
3.	Implementation	4	
4.	Design of Test Cases	4	
5.	Evaluation of Program	4	
	TOTAL	20	
Functionality of the Program is verified on the System			
Faculty Signature with Date			

```
X = model_loader.preprocessor.transform(df)
prediction = model_loader.model.predict(X)[0]
probability = (model_loader.model.predict_proba(X)[0][1])
return {
    "prediction": int(prediction)
    "churn_probability": float(probability)}
```

schemas.py:-

```
from typing import Literal
from pydantic import BaseModel, Field
class CustomerInput(BaseModel):
    customerID : str
    gender : Literal["Male", "Female"]
    SeniorCitizen: Literal[0,1]
    Partner : Literal["Yes","No"]
    Dependents : Literal["Yes","No"]
    tenure : int = Field(ge=0)
    MonthlyCharges : float= Field(ge=0)
    TotalCharges :str
```

model_loader.py:-

```
import joblib
model = joblib.load("deployment/model.pkl")
preprocessor = joblib.load("deployment/preprocessor.pkl")
```

Commands to Execute:-

```
cd "Path\to\churn-prediction
```

rebuild.py:-

```
import csv, shutil, path
root = path(__file__).resolve().parent
with open(root/"FILE-PLACEMENT_MAD.CSV',
```

```
        encoding = "utf-8-sig",
        newline = "") as file:

    reader = csv.DictReader(file)
    for row in reader:
        source = root/row["PackagedFile"]
        destination = root/path(row[
            'OriginalProjectPath"].replace("\\", "/"))
        if not source.is_file():
            print("Missing:", source.name)
            continue
        destination.parent.mkdir(
            parents=True,
            exist_ok=True)
        if source.resolve() != destination.resolve():
            shutil.copy2(source, destination)
    print("Project folders reconstructed succesfully")
```

LABORATORY CONTINUOUS EVALUATION			
S.No.	PROGRAM	Max. Marks	Marks Scored
1.	Design of Algorithm	4	
2.	Viva-Voce on Algorithm	4	
3.	Implementation	4	
4.	Design of Test Cases	4	
5.	Evaluation of Program	4	
	TOTAL	20	

Functionality of the Program is verified on the System

Faculty Signature with Date

② py -3.11 -m venve venv

③ .\venv\scripts\Activate.PS1

④ .python -m pip install --upgrade pip

⑤ pip install -r requirements -api.txt

⑥ pip install pytest httpx.

⑦ python -m uvicorn api.main:app --reload

LABORATORY CONTINUOUS EVALUATION			
S.No.	PROGRAM	Max. Marks	Marks Scored
1.	Design of Algorithm	4	
2.	Viva-Voce on Algorithm	4	
3.	Implementation	4	
4.	Design of Test Cases	4	
5.	Evaluation of Program	4	
	TOTAL	20	

Functionality of the Program is verified on the System

Faculty Signature with Date

Aim:- To containerize the customer churn prediction machine learning application using Docker and deploy it using kubernetes. To validate the containerized Fast API application, test prediction consistency, simulate deployment failures, perform debugging and recovery and demonstrate kubernetes scaling and automatic recovery.

Implementation:-

1) prepare_deployment.py :-

```
import joblib
import shutil
from pathlib import path
import joblib
import mlflow.
import mlflow.sklearn
from mlflow import MlflowClient
MODEL_NAME = "Telco_churn_Production_Model"
def get_production_model(client, model_name):
    versions= client.search_model_versions(f"name=
                {model_name}")
    production_versions = [v for v in versions.
        if v.current_stage == "Production"]
    if not production_versions:
        raise RuntimeError("No Production model
                found")
```

LABORATORY CONTINUOUS EVALUATION			
S.No.	PROGRAM	Max. Marks	Marks Scored
1.	Design of Algorithm	4	
2.	Viva-Voce on Algorithm	4	
3.	Implementation	4	
4.	Design of Test Cases	4	
5.	Evaluation of Program	4	
	TOTAL	20	

Functionality of the Program is verified on the System

Faculty Signature with Date

```
if len(production_versions) > 1:
    raise RuntimeError("Multiple Production models found")
  return Production_versions[0]
def prepare_deployment():
    project_root = path(__file__).
                   resolve().parent.
    deployment_dir = project_root / "deployment"
    deployment_dir.mkdir(parents=True, exist_ok=True)
    client = MlflowClient()
    model_version = get_Production_model(client, MODELNAME)
    model_uri = (f"models:/{MODEL_NAME}/
                 {model_version.version}")
    model = mlflow.sklearn.load_model(model_uri)
    model_path = deployment_dir / "model.pkl"
    joblib.dump(model, model_path)
    preprocessor_path = client.download_artifacts(
        model_version.run_id,
        "Preprocessing_pipeline/preprocessor.pkl",
    dst_path = str(deployment_dir))
    downloaded = path(preprocessor_path)
    final_path = deployment_dir / "preprocessor.pkl"
    if downloaded != final_path:
        shutil.move(downloaded, final_path)
    joblib.load(model_path)
    joblib.load(final_path)
```

Output :-

Docker image :
churn-prediction-api :1.0
Image created successfully
FastAPI started on port 8000
Health status : healthy
service : churn prediction API
version : 1.0.0
Model Loaded : True
Preprocessor Loaded : True
Deployment : churn-api
Initial replicas : 1
scaled replicas : 3
READY : 3/3
UP-TO-DATE : 3
AVAILABLE : 3
Failure : Missing model Artifact
Error :
File NotFound Error :
Model not Found : /app/
deployment/model.pkl
container :
Exited (3)
Recovery :
container recreated from known-good image
Result :
Application Startup successful.

LABORATORY CONTINUOUS EVALUATION			
S.No.	PROGRAM	Max. Marks	Marks Scored
1.	Design of Algorithm	4	
2.	Viva-Voce on Algorithm	4	
3.	Implementation	4	
4.	Design of Test Cases	4	
5.	Evaluation of Program	4	
	TOTAL	20	
Functionality of the Program is verified on the System			
Faculty Signature with Date			

```
metadata = {
    "model_name": MODEL_NAME,
    "model_version": str(model_version.version),
"model_stage": model_version.current_stage,
"run_id": model_version.run_id,
"source": "MLflow"}
with open(deployment_dir/"model_metadata.json", "w")
    as file: json.dump(metadata, file, indent=4)
print("Deployment preparation completed")

if __name__ == "__main__":
    prepare_deployment()
```

Dockerfile :-

```
FROM python:3.11-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONBUFFERED=1
COPY requirements-api.txt
RUN pip install --no-cache-dir -r requirements-api.txt
COPY api/. /api/
COPY deployment/. /deployment/ COPY config.yaml
EXPOSE 8000
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0",
    "--port", "8000"]
```

.dockerignore :-

```
venv/
.git/
__pycache__/
*.pyc
```

Model loaded successfully
Preprocessor loaded successfully
Initial pods: 3
One Pod deleted
Kubernetes detected the missing pod and created a replacement pod.
Final status:
READY : 3/3
AVAILABLE : 3
Service : Healthy.

LABORATORY CONTINUOUS EVALUATION			
S.No.	PROGRAM	Max. Marks	Marks Scored
1.	Design of Algorithm	4	
2.	Viva-Voce on Algorithm	4	
3.	Implementation	4	
4.	Design of Test Cases	4	
5.	Evaluation of Program	4	
	TOTAL	20	
Functionality of the Program is verified on the System			
Faculty Signature with Date			

```
.pytest-cache/
.ipynb_checkpoints/
deployment.yaml:-
api version : apps/v1
kind : Deployment
metadata :
    name : churn_api
    labels :
        app : churn_api
spec :
    selector :
        matchLabels:
            app: churn-api.
    template :
        metadata;
            labels;
                app; churn-api
        spec:
            containers:
                name: churn-api
                image :churn-prediction_api :1.0
                imagePullPolicy : IfNotPresent
                ports : containerPort : 8000
service.yaml:-
apiversion : v1
kind : service
metadata :
    name : churn-api-service
spec:
    selector :
    app: churn_api
    type : NodePort
    ports : protocol : TCP
            port : 8000
            targetport : 8000
```